

MATERI PERKULIAHAN

Software Testing dan Quality Assurance

Memastikan Kualitas Perangkat Lunak
untuk Solusi yang Andal dan Berkualitas



Test Planning



Test Execution



Bug Tracking



Quality
Assurance



Continuous
Improvement



Dosen Pengampu:

Rony Kriswibowo

Praktisi & Konsultan IT



BUILD
SOLUTIONS
GROW
TOGETHER

RKSOLUSINDO

CAPAIAN PEMBELAJARAN

CPL 05

Mampu memahami dan menggunakan berbagai metodologi pengembangan sistem beserta alat pemodelan sistem dan menganalisa kebutuhan pengguna dalam membangun sistem informasi untuk mencapai tujuan organisasi

SUB CPMK

1. Mampu Memahami berbagai metodologi pengembangan sistem
2. Mampu menggunakan berbagai alat pengembangan sistem
3. Mampu menganalisa kebutuhan pengguna dalam membangun sistem informasi untuk mencapai tujuan organisasi



SUMBER MATERI

1. Henderi, & Suryaningrat, A. (2020). Software testing as quality assurance on web application. Proceedings of the 1st International Multidisciplinary Conference on Education, Technology, and Engineering (IMCETE 2019), 199-203. Atlantis Press.
<https://doi.org/10.2991/assehr.k.200303.047>
2. Sommerville, I. (2015). Software engineering (10th ed.). Pearson
3. Kriswibowo, et al. Development of an Integrated Information System for Monitoring and Validation of Health Workers Practice Licenses (STR) in Healthcare Facilities. (2025). Jurnal Ilmiah Informatika Dan Komputer, 2(1), 48-58.
4. Rony Kriswibowo, Rusina Widha Febriana, Johan Suryo Prayogo, & Putri Ariatna Alia. (2024). Pengaruh Fitur Dan Kemudahan Penggunaan Terhadap Kepuasan Pengguna Aplikasi Si Rekap KPU. Neptunus: Jurnal Ilmu Komputer Dan Teknologi Informasi, 2(2), 16-24.
<https://doi.org/10.61132/neptunus.v2i2.82>
- 5.13. Kriswibowo, R., Rusina Widha Febriana, & Johan Suryo Prayogo. (2023). Tingkat Kebergunaan Aplikasi Pedulilindungi Mobile Menggunakan Metode Sistem Usability Scale dan Net Promoter Score: The Usability Level of Pedulilindungi Mobile Application Using the Usability Scale System and Net Promoter Score Method. Decode: Jurnal Pendidikan Teknologi Informasi, 3(1), 54-62. <https://doi.org/10.51454/decode.v3i1.120>
6. Kriswibowo R, Supriyanto B, [...] Komputer F, IJEIS (Indonesian Journal of Electronics and Instrumentation Systems) (2021) 11(1) 10.
<https://doi.org/10.22146/ijeis.63411>



OUTLINE MATERI



INTRODUCTION OF ISTQB

Mengenalkan Tujuh prinsip dasar dalam software testing



V-Model

Model V adalah proses pengembangan perangkat lunak yang menjelaskan hubungan antara setiap fase siklus hidup pengembangan dan fase pengujian yang sesuai.



4 LEVEL TESTING

Unit → Integration → System → Acceptance



STRATEGI INTEGRASI

Top-Down vs Bottom-Up



International Software Testing Qualifications Board (ISTQB)

ISTQB (International Software Testing Qualifications Board) mendefinisikan 7 prinsip dasar yang menjadi fondasi bagi seorang software tester. Memahami prinsip ini membantu dalam menetapkan ekspektasi dan strategi yang tepat.

Testing Shows the Presence of Defects

Tujuan utama software testing adalah untuk menemukan masalah/defect atau ketidaksesuaian dari perilaku yang diharapkan (yang tidak sesuai dengan produk/aplikasi).

Contohnya, ketika sebuah perusahaan teknologi membuat software baru maka perusahaan tersebut akan melakukan serangkaian test cases terkait aplikasi tersebut agar meminimalisir adanya masalah/defect dikemudian hari.

Exhaustive Testing Is Impossible

Tidak mungkin untuk menguji setiap skenario/kemungkinan secara menyeluruh. Oleh karena itu, kita harus memilih test cases yang paling penting berdasarkan risiko dan kebutuhan. Umumnya kita menerapkan teknik Boundary-Value Analysis dalam prinsip ini.

Contohnya, sebuah aplikasi web memiliki banyak fitur, dan terdapat banyak variasi browser, perangkat, dan user action. Saat melakukan testing, tidak mungkin untuk melakukan secara menyeluruh dengan bermacam kombinasi, jadi pengujian yang baik yaitu berfokus pada skenario kritis dan umum.

Early Testing

Kegiatan pengujian harus dimulai sedini mungkin dalam proses pengembangan aplikasi perangkat lunak/SDLC (Software Development Life Cycle).

Contohnya, saat menggunakan environment development Agile, testing dimulai sejak awal seperti pada tahap desain atau sesegera mungkin setelah modul dasar telah dibuat. Ini membantu dalam mengidentifikasi dan mengatasi masalah/defect sejak dini.

Defect Clustering

Sejumlah kecil modul atau fungsi pada aplikasi biasanya mengandung sebagian besar masalah/defect. Oleh sebab itu dengan memfokuskan testing pada area dengan risiko lebih tinggi atau sering terjadinya masalah/defect adalah strategi yang efektif.

Contohnya, setelah beberapa kali update, ditemukan bahwa modul tertentu pada software yang besar umumnya memiliki lebih banyak masalah/defect. Kemudian testing difokuskan pada area berisiko tinggi tersebut.

Pesticide Paradox

Jika rangkaian pengujian yang sama diulangi, efektivitas pengujian tersebut akan menurun seiring berjalannya waktu.

Contohnya, menjalankan rangkaian testing yang sama untuk setiap rilis software mungkin tidak menunjukkan masalah/defect baru. Untuk mengatasi hal ini, scenario test cases testing memerlukan pembaruan dan penambahan rutin untuk beradaptasi dengan perubahan software.

Testing Is Context-Dependent

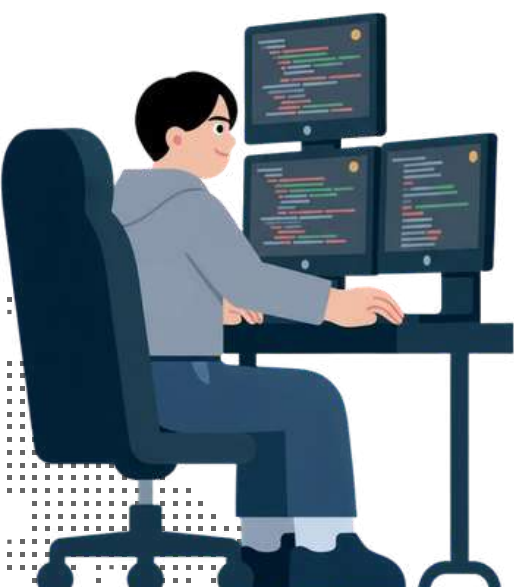
Efektivitas dari testing bergantung pada konteks penerapannya.

Contohnya, pendekatan yang dilakukan untuk pengujian aplikasi software e-commerce akan berbeda dengan software game. Pentingnya fungsi aplikasi dan ekspektasi dari user menentukan strategi pengujian.

Absence-of-Errors Fallacy

Terkadang orang berpikir jika tidak ada masalah yang ditemukan selama pengujian, berarti perangkat lunak sudah sempurna. Namun, kenyataannya, bisa saja ada masalah yang belum terdeteksi.

Contohnya, sebuah software menjalani testing secara menyeluruh, dan semua masalah/defect yang dilaporkan telah diperbaiki oleh Developer. Namun, setelah rilis, user menghadapi masalah/defect baru yang tidak teridentifikasi selama testing, sehingga menyoroti kesalahan pada asumsi bahwa produk bebas masalah/defect.



International Software Testing Qualifications Board (ISTQB) Cont..

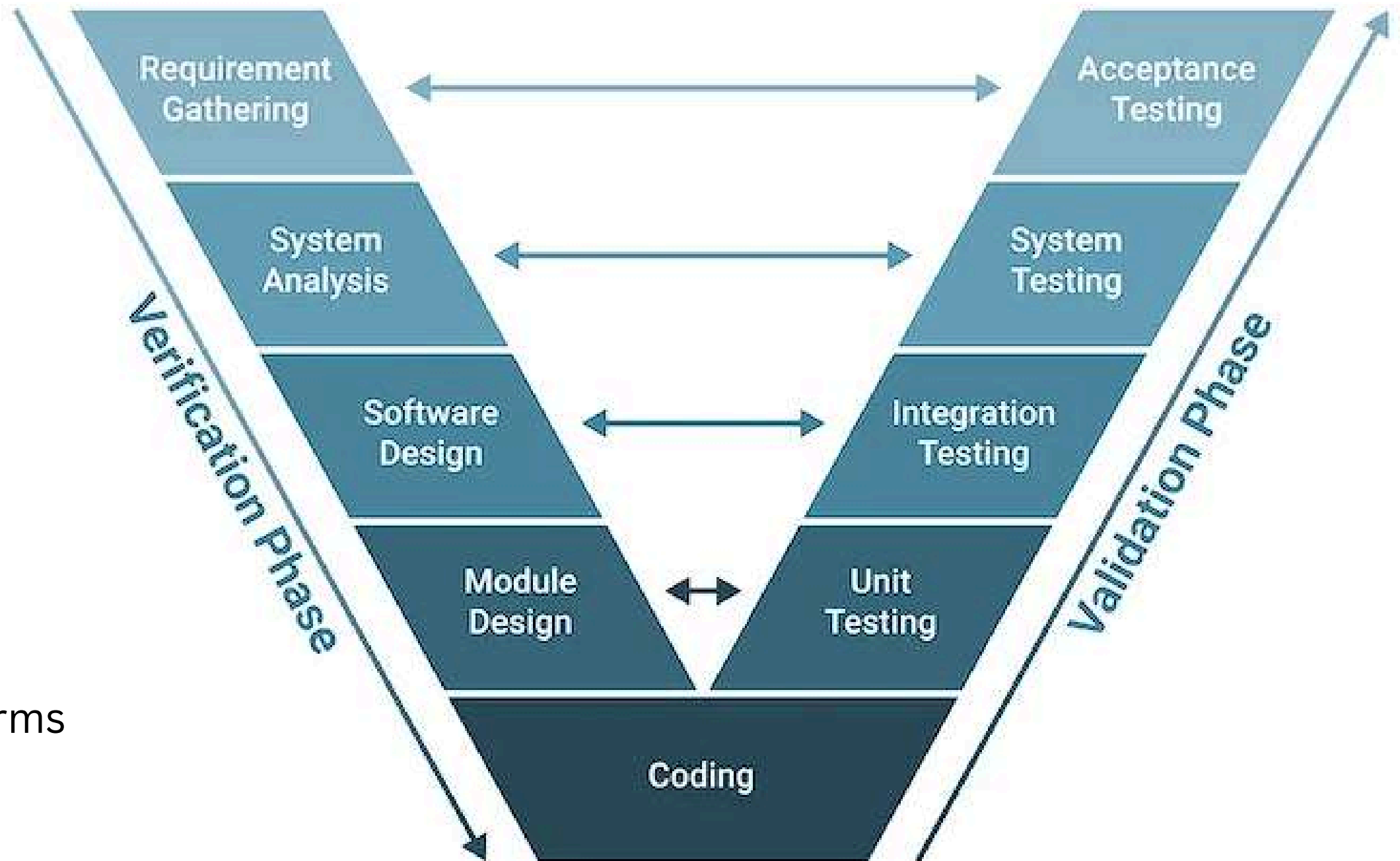
ISTQB® - FOUNDATION LEVEL					
Fundamentals of Testing	Testing Throughout the Software Development Lifecycle	Static Testing	Test Techniques	Test Management	Tool Support for Testing
What is Testing?	Software Development Lifecycle Models	Static Testing Basics	Categories of Test Techniques	Test Organisation	Test Tool Considerations
Why is Testing Necessary?	Test Levels	Review Process	Black-box Test Techniques	Test Planning and Estimation	Effective Use of Tools
Seven Testing Principles	Test Types		White-box Test Techniques	Test Monitoring and Control	
Test Process	Maintenance Testing		Experience-based Test Techniques	Configuration Management	
The Psychology of Testing				Risk and Testing	
				Defect Management	



WHAT IS THE V-MODEL IN SOFTWARE DEVELOPMENT?

The V-model is a software development process that pairs each development phase with a corresponding testing phase, emphasizing early test planning, defect prevention and clear requirements. It suits projects with well-defined scopes and prioritizes quality and documentation.

The left-hand side of the V represents the verification phase while the right-hand side represents the validation phase. Verification confirms the product is built correctly according to specifications. Validation confirms the product meets user needs and expectations.



When to Use a V-Model

V-models are used in situations wherein the requirements and understanding of the software's functionality are well-defined from the beginning. The V-model assumes stable requirements and discourages iteration, so in cases where the project scope is clear and the development team has a solid understanding of the requirements, the V-model can be an effective tool for delivering high-quality software.

✓ Advantages of Using the V-Model

- Improves Quality: From the beginning, the V-model ensures that quality is built into the development process, which results in fewer bugs in code and higher-quality software.
- Reduces Risks: The V-model provides a clear roadmap for the entire development process, which allows for better risk management and mitigation.
- Increases Efficiency: The V-model encourages collaboration between different teams and stakeholders, which results in more efficient development and testing.
- Improves Communication: The V-model emphasizes communication between stakeholders, to ensure everyone has a clear understanding of the requirements and objectives.
- Enhances Testing: The V-model places a strong emphasis on thorough and effective testing throughout the entire development process.
- Improves Documentation: The V-model requires comprehensive documentation at every stage of the development process, which leads to better record-keeping and easier code maintenance.

✗ Disadvantages of Using the V-Model

- Rigid: The V-model can be inflexible and provide very little room for changes or deviations from the plan. This rigidity can make it difficult to adapt to changing project requirements or new information.
- Time-Consuming: The V-model can be time-consuming due to its focus on thorough planning and documentation at every stage. These factors can slow down the development process and lead to longer project timelines.
- Resource Intensive: The V-model requires a significant amount of resources including time, budget and personnel, thereby making it a difficult model for small teams or organizations with limited resources to implement.
- Limited Agility: The V-model may not be well suited for Agile development approaches, which rely on flexibility, iterative development and continuous feedback.
- Overemphasis on Testing: While thorough testing is a critical component of software development, the V-Model may place too much emphasis on testing, which can lead to production delays and increased costs.



WHAT IS THE V-MODEL IN SOFTWARE DEVELOPMENT?

What are the verification and validation phases of the V-model?

Verification phases of the V-model (each aimed at ensuring the product is built correctly) include:

1. Business requirement analysis
2. System analysis
3. Software architecture design
4. Module design
5. Coding

Validation phases of the V-model (all designed to confirm the final product meets user requirements) include:

1. Unit testing
2. Integration testing
3. System testing
4. Acceptance testing



4 LEVELS OF SOFTWARE TESTING & ADVANTAGES

Before release, any software product goes through various levels of testing to make sure that it is working properly. But, generally, software testing is divided into four levels and in this article, we will focus on the 4 main levels of testing:

1. Unit testing
2. Integration testing
3. System testing
4. User acceptance testing.

4 Langkah Pengujian Software

1. UNIT TESTING

Menguji Bagian Kecil dari Kode Program



2. INTEGRATION TESTING

Menguji Penggabungan Antar Modul



3. SYSTEM TESTING

Menguji Keseluruhan Sistem



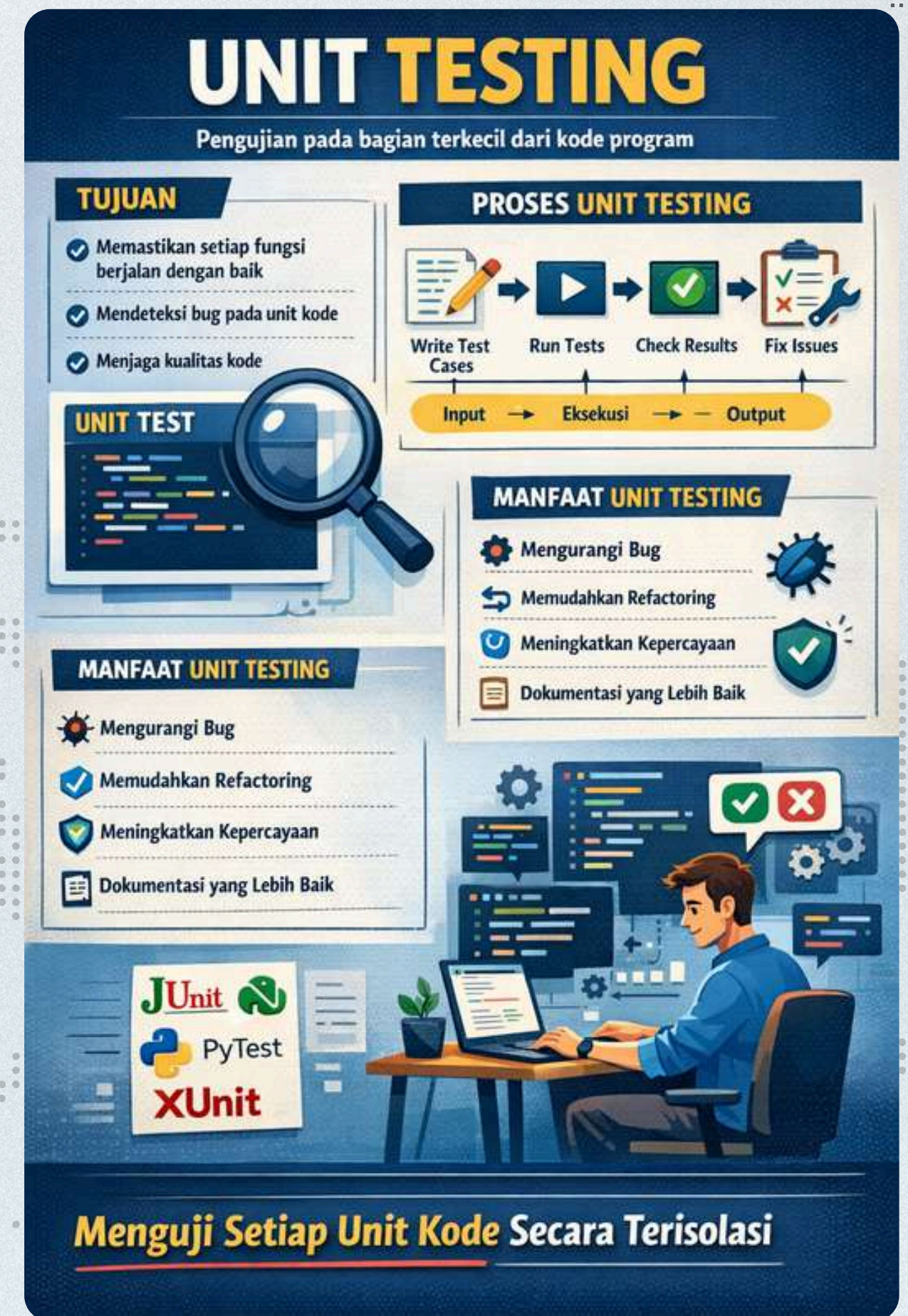
4. ACCEPTANCE TESTING

Menguji Kelayakan Sistem oleh Pengguna



Unit Testing

Unit testing (or component testing) is the first stage in the testing process. A unit is the smallest testable and independent component of the software. Normally, Unit testing is performed by software developers before handing software over to testers for executing other formal tests. The main purpose of unit testing is to make sure that every unit is working as designed. In addition, as the cost of fixing bugs in the late stages is much higher than in the early stages, unit testing helps reduce cost by finding problems early. White box testing method is normally used at this stage.

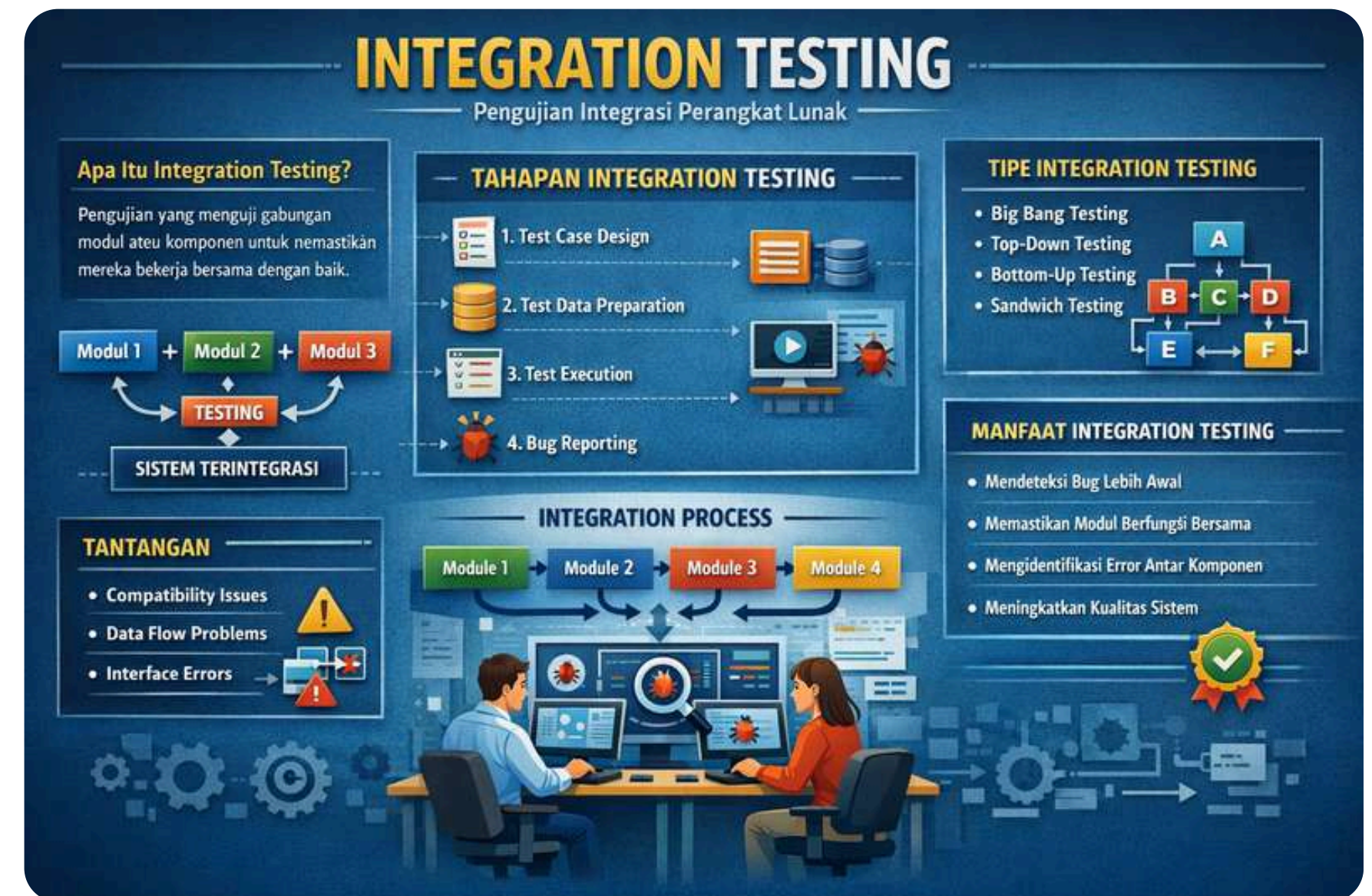


Integration Testing

After unit testing is integration testing. In this stage, all units are integrated together and tested. It is a form of testing in the testing process performed to detect defects in the interactions and the interfaces between the integrated units. Black box testing technique is usually used at this stage, but in many cases, both Black and White testing method are combined together.

There are four main integration testing approaches, namely:

- **Big bang:** In the big bang approach, All components or modules are integrated together to complete a software system. After that, integration testing is performed.
- **Top-down:** In this method, the process is carried out from the highest level modules to the lower level ones. This method requires Test stubs, Stub is a module which will temporarily substitute for submodules if they are not available for integration during the early phases.
- **Bottom-up:** Bottom-up testing is a method where the lowest level components are tested first, and then progressively higher level components are tested. Drivers are required in this method to simulate the main modules which are not available in the early phases.
- **Sandwich:** Sandwich testing (or Hybrid approach) is a type of integration testing which is the combination of Top-down and Bottom-up testing.



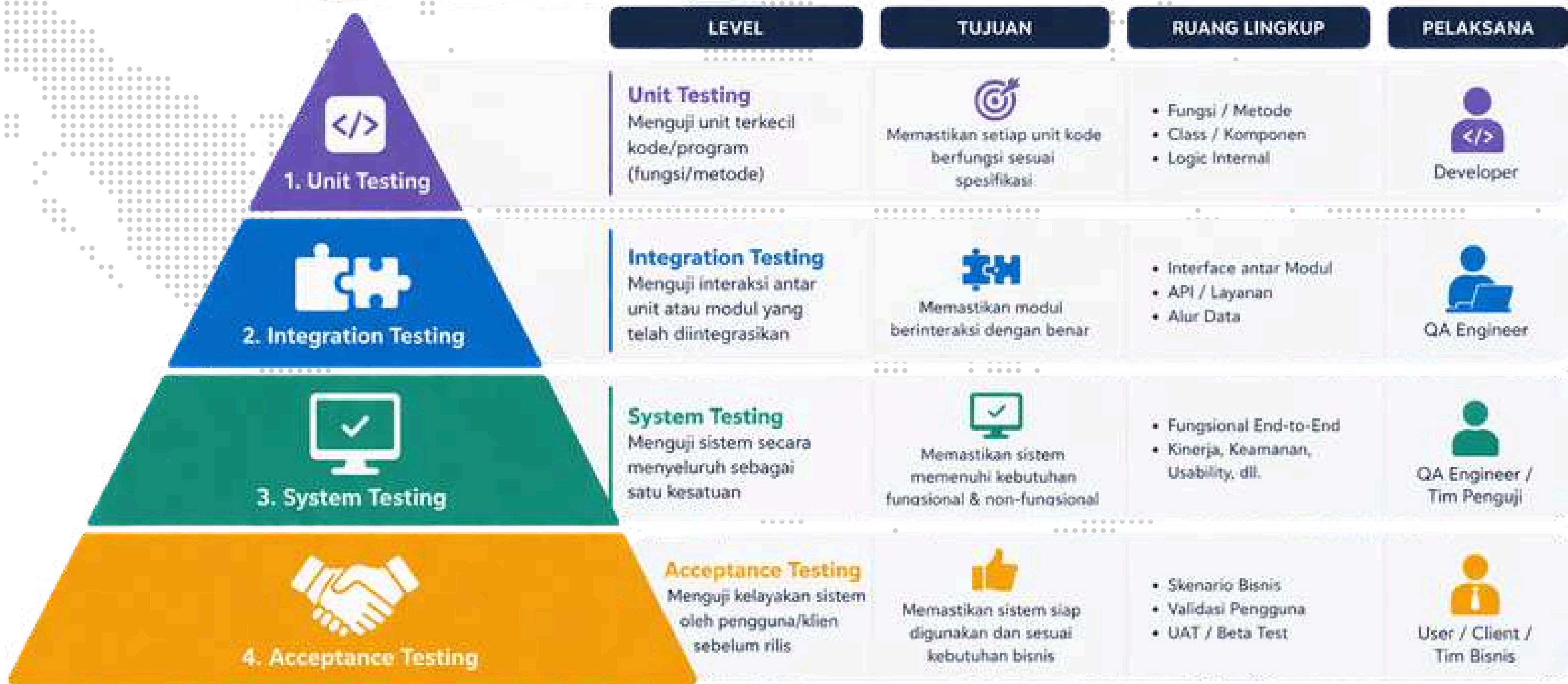
System Testing

After integration testing, the fully integrated application is tested to check that whether the system meets its software requirements specifications (SRS). System Testing is a type of black box testing method thus the knowledge of internal code is not required. This testing is performed in an environment that closely resembles production environment in order to provide more reliable and efficient outcoming. It is an important phase as it helps evaluate the functional, business and technical requirements.



HIRARKI LEVEL TESTING

Pengujian dilakukan bertahap dari unit terkecil hingga sistem secara keseluruhan untuk memastikan kualitas perangkat lunak.



Applying Top-Down in Software Design

Software is built to solve a problem or achieve a goal.

Using the top-down approach, developers begin by defining that goal clearly.

Step 1: Define the Design Goal

Start with a broad statement of what the software should do. Also clarify what it won't do—this helps set boundaries.



Design goal:

Create software that catalogs books by subject matter



Out of scope:

Personal library cataloging or custom category creation



Step 2: Break Down the Goal into Tasks

Once the goal is defined, break it into high-level tasks:



Import book data



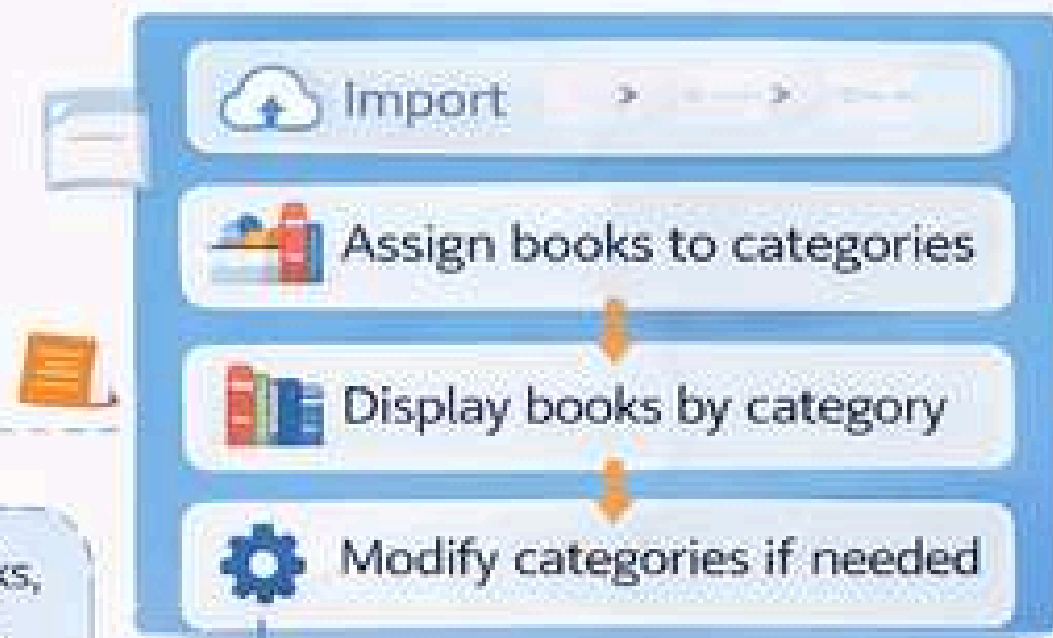
Assign books to categories



Display books by category



Each task can be further refined into subtasks, such as designing the interface or writing sorting logic.





Thank You!

See You Next Week